

Interprète

MU4IN501 – DLP : Développement d'un langage de programmation
Master STL, Sorbonne Université

Antoine Miné

Cours 2

19 septembre 2023

Année 2023–2024



Développement d'un langage de programmation :
interprète et compilateur.

Par étapes, avec ajout progressif de fonctionnalités.

- **ILP1** : langage de base

- constantes (entiers, flottants, booléens, chaînes),

- opérateurs (+, -, ×, ...),

- appels de primitives (print, ...),

- blocs locaux (let x = ... in ...),

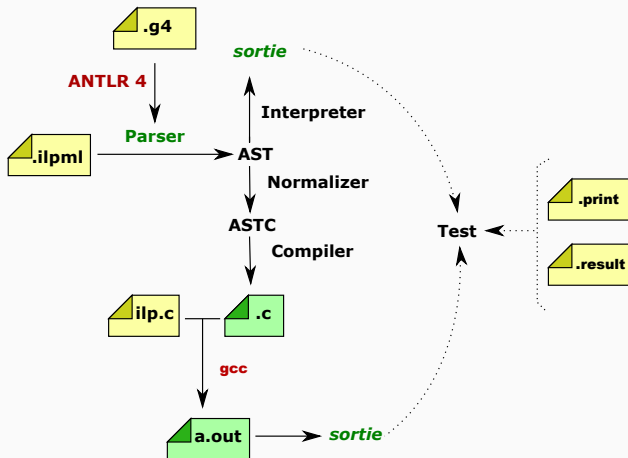
- alternatives (if ... then ... else)

- **ILP2** : ajout des boucles, affectations, fonctions globales

- **ILP3** : ajout des exceptions et fonctions de première classe

- **ILP4** : ajout des classes et des objets

Fil conducteur : schéma général d'ILP



- généralités et choix
- rappels sur l'AST
introduction du motif **visiteur**
- l'interprète ILP en Java :
 - structures de données (état de l'interprète)
 - évaluation (visiteur)
 - la bibliothèque d'exécution Java (primitives, opérateurs)
- tests JUnit 4

Modes d'interaction :

- REPL (*read-eval-print loop*) interactif (shell, OCaml, Python)
- programmes indépendants (C, Java, mais aussi shell, OCaml, Python !)

Modes d'exécution : nombreuses techniques

- interprétation directe sur chaîne (shell) (lent, limité à des grammaires simples)
- **interprétation d'AST** (utilisation d'un analyseur syntaxique, puis exécution de l'AST produit)
- interprétation de code-octet (traduction AST vers code-octet, puis exécution dans une machine virtuelle)
 - code-octet ad-hoc (Python, V8 pour JavaScript)
 - ou spécifié et portable (Java, OCaml)
- **compilation vers du binaire** (via de l'assembleur, du C, du code-octet LLVM)
- compilation *just-in-time* (compilation vers du code-octet, puis recompilation en binaire : Java, JavaScript)

Sémantique : sens donné à un programme

Spécifie comment chaque instruction :

- **modifie sa mémoire** (variables globales, locales, tas)
- **modifie l'environnement** (fichiers, communication réseaux, etc.)
- **calcule et retourne une valeur**
- **ou provoque une erreur**
- **ou a un comportement indéfini, dépendant de l'implantation**

C'est un **contrat** entre le **concepteur/implémenteur** du langage et le **programmeur**.

La sémantique peut être :

- **non-spécifiée** (Python, interprète de référence)
- **spécifiée en langage naturel** (C, Java, JavaScript)
- **décrite mathématiquement** (SML)

Interprète écrit en Java :

- prend un AST en entrée
ne se soucie donc pas de la syntaxe, ni de l'analyse syntaxique
- calcule la valeur des expressions
- exécute leurs effets (print)

Rappel, le langage ILP1 a :

- des structures de contrôle : alternatives, séquences, blocs
`if ... then ... else ..., ( ...; ... ; ... )`
- des variables locales, déclarées
`let ... = ... in ...`
- des variables et fonctions primitives
prédéfinies, globales et immuables
`pi, print, newline`
- des constantes littérales : `1, true, "Hello!"`
- des opérateurs, prédéfinis : `+, -`
- des expressions composées : `print(1 + x)`

Ces concepts existent en Java $\Rightarrow$ nous nous appuierons dessus

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

```
let x = 1 in let x = x + 1 in x
```

```
let x = 1 in let x = 2 and y = x in y
```

```
let x = print in 3
```

```
let x = print in x(3)
```

```
let true = 3 in true + 1
```

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

$\implies$ retourne 2 (portée lexicale)

```
let x = 1 in let x = x + 1 in x
```

```
let x = 1 in let x = 2 and y = x in y
```

```
let x = print in 3
```

```
let x = print in x(3)
```

```
let true = 3 in true + 1
```

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = x + 1 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = 2 and y = x in y
```

```
let x = print in 3
```

```
let x = print in x(3)
```

```
let true = 3 in true + 1
```

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = x + 1 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = 2 and y = x in y
```

⇒ retourne 1 (portée lexicale)

```
let x = print in 3
```

```
let x = print in x(3)
```

```
let true = 3 in true + 1
```

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = x + 1 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = 2 and y = x in y
```

⇒ retourne 1 (portée lexicale)

```
let x = print in 3
```

⇒ retourne 3 (une variable peut contenir une fonction)

```
let x = print in x(3)
```

```
let true = 3 in true + 1
```

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = x + 1 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = 2 and y = x in y
```

⇒ retourne 1 (portée lexicale)

```
let x = print in 3
```

⇒ retourne 3 (une variable peut contenir une fonction)

```
let x = print in x(3)
```

⇒ affiche 3 (appel indirect, via une variable, possible)

```
let true = 3 in true + 1
```

Puzzles sémantiques (1/2)

```
let x = 1 in let x = 2 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = x + 1 in x
```

⇒ retourne 2 (portée lexicale)

```
let x = 1 in let x = 2 and y = x in y
```

⇒ retourne 1 (portée lexicale)

```
let x = print in 3
```

⇒ retourne 3 (une variable peut contenir une fonction)

```
let x = print in x(3)
```

⇒ affiche 3 (appel indirect, via une variable, possible)

```
let true = 3 in true + 1
```

⇒ erreur d'analyse syntaxique (true est un mot-clé, pas un nom de variable valide)

Puzzles sémantiques (2/2)

let print = 3 in print + 1

let print = 3 in print (print)

let x = 1 + true in x

if 1 then 2 else 3

if 0 then 2 else 3

9223372036854775807 + 1 $((2^{63} - 1) + 1)$

9007199254740992. + 1. $(2^{52} + 1)$

Puzzles sémantiques (2/2)

`let print = 3 in print + 1`

$\implies$ **retourne 4** (une variable peut cacher une primitive)

`let print = 3 in print (print)`

`let x = 1 + true in x`

`if 1 then 2 else 3`

`if 0 then 2 else 3`

`9223372036854775807 + 1` ($(2^{63} - 1) + 1$)

`9007199254740992. + 1.` ($2^{52} + 1$)

Puzzles sémantiques (2/2)

let print = 3 in print + 1

⇒ retourne 4 (une variable peut cacher une primitive)

let print = 3 in print (print)

⇒ erreur d'exécution (3 n'est pas une fonction)

let x = 1 + true in x

if 1 then 2 else 3

if 0 then 2 else 3

9223372036854775807 + 1 ($(2^{63} - 1) + 1$)

9007199254740992. + 1. ($2^{52} + 1$)

Puzzles sémantiques (2/2)

`let print = 3 in print + 1`

⇒ retourne 4 (une variable peut cacher une primitive)

`let print = 3 in print (print)`

⇒ erreur d'exécution (3 n'est pas une fonction)

`let x = 1 + true in x`

⇒ erreur d'exécution (+ n'accepte pas les booléens)

`if 1 then 2 else 3`

`if 0 then 2 else 3`

`9223372036854775807 + 1` ($(2^{63} - 1) + 1$)

`9007199254740992. + 1.` ($2^{52} + 1$)

Puzzles sémantiques (2/2)

`let print = 3 in print + 1`

⇒ retourne 4 (une variable peut cacher une primitive)

`let print = 3 in print (print)`

⇒ erreur d'exécution (3 n'est pas une fonction)

`let x = 1 + true in x`

⇒ erreur d'exécution (+ n'accepte pas les booléens)

`if 1 then 2 else 3`

⇒ retourne 3 (if accepte des valeurs arbitraires)

`if 0 then 2 else 3`

`9223372036854775807 + 1` ($(2^{63} - 1) + 1$)

`9007199254740992. + 1.` ($2^{52} + 1$)

Puzzles sémantiques (2/2)

`let print = 3 in print + 1`

⇒ retourne 4 (une variable peut cacher une primitive)

`let print = 3 in print (print)`

⇒ erreur d'exécution (3 n'est pas une fonction)

`let x = 1 + true in x`

⇒ erreur d'exécution (+ n'accepte pas les booléens)

`if 1 then 2 else 3`

⇒ retourne 3 (if accepte des valeurs arbitraires)

`if 0 then 2 else 3`

⇒ retourne 3 (tout ce qui n'est pas false est vrai)

`9223372036854775807 + 1` ($(2^{63} - 1) + 1$)

`9007199254740992. + 1.` ($2^{52} + 1$)

Puzzles sémantiques (2/2)

`let print = 3 in print + 1`

⇒ retourne 4 (une variable peut cacher une primitive)

`let print = 3 in print (print)`

⇒ erreur d'exécution (3 n'est pas une fonction)

`let x = 1 + true in x`

⇒ erreur d'exécution (+ n'accepte pas les booléens)

`if 1 then 2 else 3`

⇒ retourne 3 (if accepte des valeurs arbitraires)

`if 0 then 2 else 3`

⇒ retourne 3 (tout ce qui n'est pas false est vrai)

`9223372036854775807 + 1` ($(2^{63} - 1) + 1$)

⇒ 9223372036854775808 (entiers de précision arbitraire, dans l'interprète seulement !)

`9007199254740992. + 1.` ($2^{52} + 1$)

Puzzles sémantiques (2/2)

```
let print = 3 in print + 1
```

⇒ retourne 4 (une variable peut cacher une primitive)

```
let print = 3 in print (print)
```

⇒ erreur d'exécution (3 n'est pas une fonction)

```
let x = 1 + true in x
```

⇒ erreur d'exécution (+ n'accepte pas les booléens)

```
if 1 then 2 else 3
```

⇒ retourne 3 (if accepte des valeurs arbitraires)

```
if 0 then 2 else 3
```

⇒ retourne 3 (tout ce qui n'est pas false est vrai)

```
9223372036854775807 + 1 (( $2^{63} - 1$ ) + 1)
```

⇒ 9223372036854775808 (entiers de précision arbitraire, dans l'interprète seulement!)

```
9007199254740992. + 1. ( $2^{52} + 1$ )
```

⇒ 9007199254740992. (arithmétique flottante IEEE en double-précision 64-bit)

Représentation des valeurs

Les expressions s'évaluent en **valeurs**.

Nous nous appuyons encore sur la **représentation Java** :

- booléens $\Rightarrow$ classe **Boolean**
- entiers $\Rightarrow$ classe **BigInteger** (précision arbitraire)
- flottants $\Rightarrow$ classe **Double**
- chaînes $\Rightarrow$ classe **String**

mais également des valeurs pour représenter des objets appelables, comme les primitives : voir l'interface **Invocable** plus loin

Notes :

- pas d'utilisation des types primitifs Java (int, boolean, ...)
- il faut un **type commun** capable de **représenter n'importe quelle valeur** pour typer en Java les variables ILP (de type dynamique)
 $\Rightarrow$ choix d'**Object** en Java
- **test dynamique** de type parfois nécessaire dans l'interprète, avec **instanceof**
 $\Rightarrow$ ILP a également une primitive **type_of** (retourne une chaîne)
- toute valeur est **convertible en chaîne** (**toString** en Java)
 $\Rightarrow$ utile pour implanter **print** et **to_string** en ILP

Environnement :

associe une valeur (ou une fonction à exécuter) à chaque variable ou opérateur

- **environnement global** : fixé avant l'exécution de l'interprète (`pi`, `print`, `+`, etc.)
- **environnement local** : évolue durant l'interprétation
 - ajout de variables lors d'un `let`
 - suppression automatique en fin de bloc
 - une variable récente masque les anciennes variables de même nom

⇒ structure de **pile** d'associations

Évaluation :

à chaque type d'AST correspond une méthode `eval` :

`valeur = ast.eval(environnement)`

- réursive sur l'AST
 - évalue les opérandes avant l'opération
 - `( a; b; c )` : évalue des séquences d'expressions dans l'ordre
 - `let v = ... in ...` : évalue l'initialisation avant le corps
 - `if ... then ... else ...` : évalue la condition, puis une des deux branches
- retourne toujours une valeur
- peut éventuellement avoir un *effet de bord*

- rappels d'AST et ajout des visiteurs
 - `ANTLRGrammars/`
 - `com.paracamplus.ilp1.interfaces`
 - `com.paracamplus.ilp1.ast`
- interprète
 - `com.paracamplus.ilp1.interpreter.interfaces`
 - `com.paracamplus.ilp1.interpreter`
- bibliothèque d'exécution de l'interprète
 - `com.paracamplus.ilp1.interpreter.operator`
 - `com.paracamplus.ilp1.interpreter.primitive`
- tests JUnit 4
 - `com.paracamplus.ilp1.interpreter.test`
 - `com.paracamplus.ilp1.test`
 - `SamplesILP1/`

Rappels : grammaire ANTLR et AST

Rappels : grammaire ANTLR d'ILP1

ILPMLgrammar1.g4

```
grammar ILPMLgrammar1;
@header { package antlr4; }

// Règles syntaxiques
prog returns [com.paracampus.ilp1.interfaces.IASTprogram node]
    : (exprs+=expr ';'?) * EOF
    ;
expr returns [com.paracampus.ilp1.interfaces.IASTexpression node]
    : '(' exprs+=expr (';'? exprs+=expr)* ';'?) # Sequence
    | fun=expr '(' args+=expr? (',' args+=expr)* ')' # Invocation
    | op=('-' | '!'') arg=expr # Unary
    | arg1=expr op=('*' | '/' | '%') arg2=expr # Binary
    | arg1=expr op=('+' | '-') arg2=expr # Binary
    | arg1=expr op=('<' | '<=' | '>' | '>=') arg2=expr # Binary
    | ...
    | 'let' vars+=IDENT '=' vals+=expr ('and' vars+=IDENT '=' vals+=expr)*
    | 'in' body=expr # Binding
    | 'if' condition=expr 'then' consequence=expr
    | ('else' alternant=expr)? # Alternative
    ;

// Règles lexicales
IDENT : [a-zA-Z_] [a-zA-Z0-9_]*;
INT : [0-9]+;
...
SPACE : [ \t\r\n]+ -> skip;
```

Rappels : représentation de l'AST

IASTAlternative.java

```
public interface IASTAlternative extends IASTExpression {  
    IASTExpression getCondition();  
    IASTExpression getConsequence();  
    @Nullable IASTExpression getAlternant();  
    boolean isTernary();  
}
```

ASTAlternative.java

```
public class ASTAlternative extends ASTExpression implements IASTAlternative {  
    public ASTAlternative  
        (IASTExpression condition, IASTExpression consequence, IASTExpression alternant )  
    { this.condition = condition; this.consequence = consequence; this.alternant = alternant; }  
  
    private final IASTExpression condition;  
    private final IASTExpression consequence;  
    private @Nullable final IASTExpression alternant;  
  
    @Override public IASTExpression getCondition() { return condition; }  
    @Override public IASTExpression getConsequence() { return consequence; }  
    @Override @Nullable public IASTExpression getAlternant() { return alternant; }  
    @Override public boolean isTernary() { return this.alternant != null; }  
}
```

- simples classes « conteneurs », immuables avec accesseurs
- il manque encore l'interface des visiteurs...

IASTfactory.java

```
public interface IASTfactory {  
  
    IASTexpression newSequence(IASTexpression[] asts);  
    IASTexpression newAlternative(  
        IASTexpression condition, IASTexpression consequence, IASTexpression alternant);  
    ...  
}
```

ASTfactory.java

```
public class ASTfactory implements IASTfactory {  
  
    @Override public IASTsequence newSequence(IASTexpression[] asts)  
    { return new ASTsequence(asts); }  
  
    @Override public IASTalternative newAlternative(  
        IASTexpression condition, IASTexpression consequence, IASTexpression alternant)  
    { return new ASTalternative(condition, consequence, alternant); }  
    ...  
}
```

Programmer vis à vis des **interfaces d'AST**, pas des classes,
y compris pour la création des nœuds.

Rappels : listener ANTLR pour la construction de l'AST

ILPMLListener.java

```
package com.paracamplus.ilp1.parser.ilpml;

public class ILPMLListener implements ILPMLGrammarListener {

    protected IASTfactory factory;
    public ILPMLListener(IASTfactory factory)
    { this.factory = factory; }

    @Override public void enterAlternative(AlternativeContext ctx) { }

    @Override public void exitAlternative(AlternativeContext ctx) {
        ctx.node = factory.newAlternative(
            ctx.condition.node, ctx.consequence.node,
            (ctx.alternant == null ? null : ctx.alternant.node)
        );
    }
    ...
}
```

Problème

Où et comment ajouter l'évaluation des AST ?

Les visiteurs : généralités

Exemple : expressions simplifiées

Expr.java

```
public interface IExpr {  
    int eval();  
}
```

Const.java

```
public class Const implements IExpr {  
    private int c;  
    public Const(int x) { c = x; }  
    public int eval() { return c; }  
}
```

Add.java

```
public class Add implements IExpr {  
    private IExpr e1, e2;  
    public Add(IExpr a1, IExpr a2) { e1 = a1; e2 = a2; }  
    public int eval() { return e1.eval() + e2.eval(); }  
}
```

Expressions simples, avec évaluation récursive (motif Composite).

L'évaluation est définie par une méthode `eval` dans chaque classe.

Ajout d'un traitement

IExpr.java

```
public interface IExpr {  
    int eval();  
    String print();  
}
```

Const.java

```
public class Const implements IExpr {  
    private int c;  
    public Const(int x) { c = x; }  
    public int eval() { return c; }  
    public String print()  
    { return "" + c; }  
}
```

Add.java

```
public class Add implements IExpr {  
    private IExpr e1, e2;  
    public Add(IExpr a1, IExpr a2) { e1 = a1; e2 = a2; }  
    public int eval() { return e1.eval() + e2.eval(); }  
    public String print()  
    { return e1.print() + "+" + e2.print(); }  
}
```

Ajout d'une nouvelle fonctionnalité : l'affichage.

Solution : ajout d'une méthode `print`.

Inconvénients : peu lisible, difficilement extensible

- traitement distribué sur toutes les classes
- devoir modifier (*toutes*) les classes existantes

Vers un regroupement des traitements

IVisitor.java

```
public interface IVisitor<T> {  
    T visitConst(Const c);  
    T visitAdd(Add a);  
}
```

Eval.java

```
public class Eval implements IVisitor<Integer> {  
  
    Integer visitConst(Const c)  
    { return c.getC(); }  
  
    Integer visitAdd(Add a)  
    { return visit?(a.getE1()) + visit?(a.getE2()); }  
}
```

Print.java

```
public class Print implements IVisitor<String> {  
  
    String visitConst(Const c)  
    { return "" + c.getC(); }  
  
    String visitAdd(Add a)  
    { return visit?(a.getE1()) + "+" + visit?(a.getE2()); }  
}
```

- pour chaque opération, les cas d'AST sont regroupés dans une classe, hors AST
- ajout d'une opération = ajout d'une classe sans modification des classes d'AST !
- le nœud AST doit maintenant être passé en argument aux méthodes (getters ajoutés)
- les classes de traitement obéissent à une même interface IVisitor... modulo le type de retour (classes génériques, passage de int à Integer)

Problème !

IVisitor.java

```
public interface IVisitor<T> {  
    T visitConst(Const c);  
    T visitAdd(Add a);  
}
```

Eval.java

```
public class Eval implements IVisitor<Integer> {  
  
    Integer visitConst(Const c)  
    { return c.getC(); }  
  
    Integer visitAdd(Add a)  
    { return visit?(a.getE1()) + visit?(a.getE2()); }  
}
```

Print.java

```
public class Print implements IVisitor<String> {  
  
    String visitConst(Const c)  
    { return "" + c.getC(); }  
  
    String visitAdd(Add a)  
    { return visit?(a.getE1()) + "+" + visit?(a.getE2()); }  
}
```

Mais comment gérer les **appels récursif** du traitement sur les sous-nœuds ?

visit? doit orienter vers la bonne méthode **visitXXX** de la classe de traitement !

- mauvaise solution : **visit?** implanté comme une longue séquence de **instanceof** (dispatch « à la main »)
- bonne solution : **demandeur au sous-nœud** car il connaît son type (dispatch dynamique) mais le sous-nœud doit rester indépendant de la classe de traitement. . .

Solution : *double dispatch*

IExpr.java

```
public interface IExpr {  
    <T> T accept(IVisitor<T> v);  
}
```

IVisitor.java

```
public interface IVisitor<T> {  
    T visitConst(Const c);  
    T visitAdd(Add a);  
}
```

Eval.java

```
public class Eval implements IVisitor<Integer> {  
  
    Integer visitConst(Const c)  
    { return c.getC(); }  
  
    Integer visitAdd(Add a)  
    { return a.getE1().accept(this) + a.getE2().accept(this); }  
}
```

Add.java

```
public class Add implements IExpr {  
    private IExpr e1, e2;  
    public Add(IExpr a1, IExpr a2)  
    { e1 = a1; e2 = a2; }  
    public IExpr getE1() { return e1; }  
    public IExpr getE2() { return e2; }  
    public <T> T accept(IVisitor<T> v)  
    { return v.visitAdd(this); }  
}
```

Ajout d'une méthode `accept` à chaque type d'AST (unique, commune à tous les visiteurs)

Étant donné `IExpr a = new Add(e1, e2)`, `a.accept(new Eval())` va :

- appeler la méthode `accept` de `Add`
- qui appelle `visitAdd`
 - `a.getE1().accept(this)` appelle la méthode `accept` de `e1`
 - qui appelle la bonne méthode `visit` en fonction du type de `e1` !
 - etc.

Une petite simplification : surcharge de méthodes

IExpr.java

```
public interface IExpr {  
    <T> T accept(IVisitor<T>);  
}
```

IVisitor.java

```
public interface IVisitor<T> {  
    T visit(Const c);  
    T visit(Add a);  
}
```

Eval.java

```
public class Eval implements IVisitor<Integer> {  
  
    Integer visit(Const c)  
    { return c.getC(); }  
  
    Integer visit(Add a)  
    { return a.getE1().accept(this) + a.getE2().accept(this); }  
}
```

Add.java

```
public class Add implements IExpr {  
    private IExpr e1, e2;  
    public Add(IExpr a1, IExpr a2)  
    { e1 = a1; e2 = a2; }  
    public IExpr getE1() { return e1; }  
    public IExpr getE2() { return e2; }  
    public <T> T accept(IVisitor<T> v)  
    { return v.visit(this); }  
}
```

Surcharge de la méthode visit pour des types de nœuds différents

⇒ Java trouvera automatiquement (à la compilation) la bonne signature. . .

La méthode accept est maintenant identique dans tous les nœuds !

Visiteurs de nœuds d'AST

Interface de nœud visitable

IASTvisitable.java

```
package com.paracampus.ilp1.interfaces;

public interface IASTvisitable {
    <Result, Data, Anomaly extends Throwable>
    Result accept(IASTvisitor<Result, Data, Anomaly> visitor,
                  Data data) throws Anomaly;
}
```

IASTexpression.java

```
public abstract interface IASTexpression extends IAST, IASTvisitable {
    // vide !
}
```

Les nœuds d'expression héritent de `IASTvisitable` qui impose la présence d'`accept`

Le visiteur est paramétré par :

- un type d'`argument` « données » `Data`
- le type de `retour` `Result`
- les éventuelles `erreurs` `Anomaly`

qui dépendront beaucoup du type de traitement fait par le visiteur

IASTvisitor.java

```
public interface IASTvisitor<Result, Data, Anomaly extends Throwable>
{
    Result visit(IASTalternative iast, Data data) throws Anomaly;
    Result visit(IASTbinaryOperation iast, Data data) throws Anomaly;
    Result visit(IASTblock iast, Data data) throws Anomaly;
    Result visit(IASTboolean iast, Data data) throws Anomaly;
    Result visit(IASTfloat iast, Data data) throws Anomaly;
    Result visit(IASTinteger iast, Data data) throws Anomaly;
    Result visit(IASTinvocation iast, Data data) throws Anomaly;
    Result visit(IASTsequence iast, Data data) throws Anomaly;
    Result visit(IASTstring iast, Data data) throws Anomaly;
    Result visit(IASTunaryOperation iast, Data data) throws Anomaly;
    Result visit(IASTvariable iast, Data data) throws Anomaly;
}
```

Une méthode **visit** par type de nœud...

toujours programmée vis à vis des **interfaces** des nœuds, par de leurs classes !

Exemple de nœud complet

ASTAlternative.java

```
public class ASTAlternative extends ASTExpression implements IASTAlternative {

    public ASTAlternative
    (IASTExpression condition, IASTExpression consequence, IASTExpression alternant )
    { this.condition = condition; this.consequence = consequence; this.alternant = alternant; }

    private final IASTExpression condition;
    private final IASTExpression consequence;
    private @Nullable final IASTExpression alternant;

    @Override public IASTExpression getCondition() { return condition; }
    @Override public IASTExpression getConsequence() { return consequence; }
    @Override @Nullable public IASTExpression getAlternant() { return alternant; }
    @Override public boolean isTernary() { return this.alternant != null; }

    @Override public <Result, Data, Anomaly extends Throwable>
    Result accept(IASTvisitor<Result, Data, Anomaly> visitor, Data data) throws Anomaly
    { return visitor.visit(this, data); }
}
```

Toutes les méthodes `accept` sont similaires !

Extensibilité dans deux directions

		interprète	compilateur	
		Interpreter	GlobalVariableCollector	Compiler
ILP1	IASTalternative	visit(...)	visit(...)	visit(...)
	IASTblock	visit(...)	visit(...)	visit(...)
	IASTboolean	visit(...)	visit(...)	visit(...)
	IASTfloat	visit(...)	visit(...)	visit(...)
	IASTinteger	visit(...)	visit(...)	visit(...)
	IASTstring	visit(...)	visit(...)	visit(...)
	IASTinvocation	visit(...)	visit(...)	visit(...)
	IASTbinaryOperation	visit(...)	visit(...)	visit(...)
	IASTunaryOperation	visit(...)	visit(...)	visit(...)
	IASTsequence	visit(...)	visit(...)	visit(...)
ILP2	IASTloop	visit(...)	visit(...)	visit(...)
	IASTassignment	visit(...)	visit(...)	visit(...)

Aujourd'hui : visiteur d'interprète sur ILP1,
mais dans la suite, il faudra considérer **deux** types d'extension
(toujours sans modifier le code existant)

- cours 3 et 4 : **compilation ILP1** $\Rightarrow$ **ajout de visiteurs**
facile : les visiteurs sont faits pour
- cours 5 : **ILP2** $\Rightarrow$ **ajout de nœuds AST** dans des visiteurs existants
nous verrons que c'est possible avec de l'héritage et des `cast`
mais pas très élégant... il n'existe pas vraiment de solution parfaite

Interprète

IGlobalVariableEnvironment.java

```
package com.paracampus.ilp1.interpreter.interfaces;

public interface IGlobalVariableEnvironment {
    Object getGlobalVariableValue (String variableName);
    void addGlobalVariableValue (String variableName, Object value);
    void addGlobalVariableValue (IPrimitive primitive);
    void updateGlobalVariableValue (String variableName, Object value);
}
```

Association globale entre nom de variable **String** et sa valeur **Object** :

- **getGlobalVariableValue** : **retrouver** la valeur d'une variable
- **addGlobalVariableValue** : **ajouter** une variable
en ILP1, seulement pendant l'initialisation, avant l'évaluation
- cas particulier pour les primitives car elles connaissent leur nom
inutile donc de préciser `variableName...` (cf interface **IPrimitive** un peu plus tard)
- **updateGlobalVariableValue** : modifier la valeur d'une variable
utilisé seulement à partir d'ILP2...

L'environnement global : implantation

GlobalVariableEnvironment.java

```
package com.paracampus.ilp1.interpreter;

public class GlobalVariableEnvironment implements IGlobalVariableEnvironment {

    private final Map<String, Object> globalVariableEnvironment;

    public GlobalVariableEnvironment ()
    { this.globalVariableEnvironment = new HashMap<>(); }

    @Override public Object getGlobalVariableValue(String variableName)
    { return globalVariableEnvironment.get(variableName); }

    @Override public void addGlobalVariableValue(String variableName, Object value)
    { globalVariableEnvironment.put(variableName, value); }

    @Override public void addGlobalVariableValue(IPrimitive primitive)
    { globalVariableEnvironment.put(primitive.getName(), primitive); }

    @Override public void updateGlobalVariableValue(String variableName, Object value)
    { globalVariableEnvironment.put(variableName, value); }
}
```

Simple délégation à une [HashMap](#) Java !

EvaluationException.java

```
public class EvaluationException extends Exception
{
    public EvaluationException(String msg) { super(msg); }
    public EvaluationException(Exception e) { super(e); }
}
```

Exception générique de l'interprète pour signaler une erreur :

- erreur de l'interprète, avec un message
- ou une exception Java signalée pendant l'exécution et empaquetée dans une `EvaluationException` avant d'être retransmise (division par zéro, etc.)

ASTnamed.java

```
public abstract class ASTnamed extends AST implements Inamed
{
    private String name;
    public ASTnamed(String name) { this.name = name; }
    @Override public String getName() { return name; }
}
```

ASTvariable.java

```
public class ASTvariable extends ASTnamed implements IASTvariable
{
    public ASTvariable (String name) { super(name); }

    @Override public <Result, Data, Anomaly extends Throwable>
    Result accept(IASTvisitor<Result, Data, Anomaly> visitor, Data data) throws Anomaly
    { return visitor.visit(this, data); }
}
```

- simple conteneur pour un nom de variable, `String`, fixé à la construction
- code factorisé dans une classe `ASTnamed` utilitaire
- et bien sûr le visiteur

L'environnement local (lexical) : interface

IEnvironment.java

```
package com.paracampus.ilpl.interfaces;

public interface IEnvironment<K,V,T extends Throwable> {
    boolean isPresent(K key);
    IEnvironment<K,V,T> extend(K key, V value);
    K getKey() throws T;
    V getValue(K key) throws T;
    void update(K key, V value) throws T;
    boolean isEmpty();
    IEnvironment<K,V,T> getNext() throws T;
}
```

IEnvironment : structure générique

- **K** : type des clé
- **V** : type des valeurs
- **T** : erreur signalée par une recherche si la clé n'est pas trouvée

L'environnement local (lexical) : interface

Environment.java

```
package com.paracamplus.ilpl.interfaces;
public interface IEnvironment<K,V,T extends Throwable> {
    boolean isPresent(K key);
    IEnvironment<K,V,T> extend(K key, V value);
    K getKey() throws T;
    V getValue(K key) throws T;
    void update(K key, V value) throws T;
    boolean isEmpty();
    IEnvironment<K,V,T> getNext() throws T;
}
```

C'est une **liste chaînée**, chaque **IEnvironment** est un maillon :

- **extend** : ajout d'un maillon **en tête** de liste, qui est retourné
- **isEmpty** : si on est en fin de liste
- **getKey** : clé du maillon courant
- **getNext** : maillon suivant
- **getValue** : recherche la clé dans le maillon et **tous les suivants** retourne la première valeur trouvée, ou une exception si non trouvé
- **isPresent** : si la clé est présente (recherche dans tous les maillons)
- **update** : modifie la valeur de la clé (recherche dans tous les maillons, s'arrête au premier)

L'environnement local (lexical) : interface

IEnvironment.java

```
package com.paracampus.ilp1.interfaces;

public interface IEnvironment<K,V,T extends Throwable> {
    boolean isPresent(K key);
    IEnvironment<K,V,T> extend(K key, V value);
    K getKey() throws T;
    V getValue(K key) throws T;
    void update(K key, V value) throws T;
    boolean isEmpty();
    IEnvironment<K,V,T> getNext() throws T;
}
```

ILexicalEnvironment.java

```
package com.paracampus.ilp1.interpreter.interfaces;

public interface ILexicalEnvironment
    extends IEnvironment<IASTvariable, Object, EvaluationException> {
    @Override ILexicalEnvironment extend(IASTvariable variable, Object value);
    @Override ILexicalEnvironment getNext() throws EvaluationException;
}
```

ILexicalEnvironment : spécialisation

- les clés sont des **IASTvariable**
- les valeurs sont des **Object**
- **EvaluationException** : erreur générique de l'interprète

L'environnement local (lexical) : implantation, maillon

LexicalEnvironment.java

```
public class LexicalEnvironment implements ILexicalEnvironment {

    private final IASTvariable variable;
    private Object value;
    private final ILexicalEnvironment next;

    public LexicalEnvironment
    (IASTvariable variable, Object value, ILexicalEnvironment next )
    { this.variable = variable; this.value = value; this.next = next; }

    @Override public ILexicalEnvironment extend(IASTvariable variable, Object value)
    { return new LexicalEnvironment(variable, value, this); }

    @Override public Object getValue(IASTvariable key) throws EvaluationException {
    if ( key.getName().equals(getKey().getName()) ) return getValue();
        else return getNext().getValue(key);
    }

    @Override public void update(IASTvariable key, Object value) throws EvaluationException {
        if ( key.getName().equals(getKey().getName()) ) updateValue(value);
        else getNext().update(key, value);
    }
    ...
}
```

Maillon de liste chaînée.

L'environnement local (lexical) : implantation, cas vide

EmptyEnvironment.java

```
public class EmptyEnvironment implements ILexicalEnvironment {

    public EmptyLexicalEnvironment() {}

    @Override public ILexicalEnvironment extend(IASTvariable variable, Object value)
    { return new LexicalEnvironment(variable, value, this); }

    @Override public Object getValue(IASTvariable key) throws EvaluationException
    { throw new EvaluationException("No such variable " + key.getName()); }

    @Override public void update(IASTvariable key, Object value) throws EvaluationException
    { throw new EvaluationException("Empty environment"); }

    ...
}
```

Cas vide.

Un environnement lexical est un chaîne de maillons `LexicalEnvironment` qui se termine par un `EmptyEnvironment`.

(modèle : listes à la OCaml ou LISP)

L'environnement des opérateurs

IOperatorEnvironment.java

```
package com.paracampus.ilp1.interpreter.interfaces;

public interface IOperatorEnvironment
{
    IOperator getUnaryOperator (IASTOperator operator) throws EvaluationException;
    IOperator getBinaryOperator (IASTOperator operator) throws EvaluationException;
    void addOperator (IOperator operator) throws EvaluationException;
}
```

Environnement global des opérateurs :

- **IASTOperator** est un Inamed, comme IASTvariable...
- distinction nécessaire entre opérateurs **unaires** et **binaires** (arité)
 - nœuds AST distincts
 - un même IASTOperator peut représenter un opérateur unaire ou binaire, avec des sémantiques différentes (exemple : -)
- IOperator porte le nom, l'arité, et la sémantique
cf. bibliothèque d'exécution un peu plus tard
- implantation **OperatorEnvironment** très similaire à **GlobalVariableEnvironment**, avec des HashMap
(mais pas de update, seulement add et get)

Interpreter.java (fragment)

```
public class Interpreter
implements IASTvisitor<Object, ILexicalEnvironment, EvaluationException> {

    protected IGlobalVariableEnvironment globalVariableEnvironment;
    protected IOperatorEnvironment operatorEnvironment;
    public Interpreter(IGlobalVariableEnvironment globalVariableEnvironment,
                      IOperatorEnvironment operatorEnvironment ) {
        this.globalVariableEnvironment = globalVariableEnvironment;
        this.operatorEnvironment = operatorEnvironment;
    }

    public Object visit(... ast, ILexicalEnvironment lexenv) throws EvaluationException
    { ... }
}
```

L'interprète est un **visiteur**, chaque **type d'AST** a une méthode **visit** qui :

- retourne une valeur ILP, représentée par un **Object** Java
- ou une exception **EvaluationException**
- **prend en argument l'environnement lexical lexenv**
⇒ il est **local** à chaque nœud, et changera lors des appels récursifs aux fils !
- a des environnements globaux et d'opérateurs **fixés à la construction**

ASTconstant.java

```
public abstract class ASTconstant
extends ASTexpression implements IASTconstant
{
    private final String description;
    private final Object value;

    public ASTconstant(String desc, Object val) ...

    @Override public String getDescription() ...
    @Override public Object getValue() ...
}
```

ASTinteger.java

```
public abstract class ASTinteger
extends ASTconstant implements IASTinteger
{
    public ASTinteger (String desc)
    { super(desc, new BigInteger(desc)); }

    ...
}
```

Interpreter.java (suite)

```
@Override public Object visit(IASTinteger iast, ILexicalEnvironment lexenv) throws EvaluationException
{ return iast.getValue(); }
```

ASTconstant factorise le code commun à tous les littéraux.

Chaque littéral a déjà converti sa représentation **chaîne** (texte du source) en valeur Java **Object** utilisable par l'interprète.

Interpreter.java (suite)

```
@Override public Object visit(IASTvariable iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    try { return lexenv.getValue(iast); }
    catch (EvaluationException exc)
    { return getGlobalVariableEnvironment().getGlobalVariableValue(iast.getName()); }
}
```

Recherche de la valeur d'une variable :

- recherche en priorité dans l'**environnement lexical**
- puis dans l'**environnement global** (si non trouvé)
- puis **signallement d'erreur** (si toujours pas trouvé)
(l'erreur est signalée par getGlobalVariableValue et propagée)

Implante donc les règles de **portée** classique. . .

Les erreurs de variables non définies sont dynamiques,
signalées uniquement si le nœud variable est exécuté

(comme en Python, à l'opposé de C et Java)

Interpreter.java (suite)

```
@Override public Object visit(IASTsequence iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    IASTexpression[] expressions = iast.getExpressions();
    Object lastValue = null;
    for ( IASTexpression e : expressions ) lastValue = e.accept(this, lexenv);
    return lastValue;
}
```

Évalue les nœuds de la séquence dans l'ordre :

- appel **récurif** sur le visiteur `e.accept(this, lexenv)`
- retourne la valeur retournée par la dernière expression
(une séquence ne peut pas être vide, `null` n'est donc jamais retourné)
- peut être interrompu par une exception...

À retenir

ne pas oublier les **appels récurifs** aux fils ;
appeler `accept`, pas directement `visit`

```
private static Object whatever = "whatever";

@Override public Object visit(IASTAlternative iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    Object c = iast.getCondition().accept(this, lexenv);
    if ( c != null && c instanceof Boolean ) {
        Boolean b = (Boolean) c;
        if ( b.booleanValue() ) return iast.getConsequence().accept(this, lexenv);
        else if ( iast.isTernary() ) return iast.getAlternant().accept(this, lexenv);
        else return whatever;
    }
    else return iast.getConsequence().accept(this, lexenv);
}
```

- il faut d'abord **évaluer la condition** :
appel récursif à **accept**, qui retourne une **valeur**
- si c'est le booléen **Boolean.TRUE**, ou si c'est un non-boléen
getConsequence est évalué (et sa valeur retournée)
- sinon, **getAlternant** est évalué (et sa valeur retournée)
- si pas d'alternant, une valeur arbitraire est retournée
(ici, **whatever**, qui est une chaîne; c'est bien un objet ILP valide)

```
@Override public Object visit(IASTbloc iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    ILexicalEnvironment lexenv2 = lexenv;
    for ( IASTbinding binding : iast.getBindings() )
    {
        Object initialisation = binding.getInitialisation().accept(this, lexenv);
        lexenv2 = lexenv2.extend(binding.getVariable(), initialisation);
    }
    return iast.getBody().accept(this, lexenv2);
}
```

Création d'un nouvel environnement lexical, `lexenv2` :

- à partir de `lexenv`
- enrichi avec la valeur des nouvelles variables
après évaluation de leur initialiseur dans l'environnement original `lexenv`
- le corps est évalué dans le nouvel environnement `lexenv2` (et sa valeur retournée)
- le nouvel environnement est jeté (l'évaluation continue avec `lexenv`)

À retenir

bien déterminer l'**environnement lexical** de `visit`

exemple

```
let x = pi+1 in let x = 2 and y = x+3 in x+y
```

code évalué	environnement lexical	valeur
let x = pi+1 in ...	[]	4.14
... let x = 2 and y = x+3 in ...	[x=4.14]	2
... let x = 2 and y = x+3 in ...	[x=4.14]	7.14
... x+y	[x=2] → [y=7.14] → [x=4.14]	9.14

La structure de pile en liste chaînée permet bien la gestion de la portée lexicale, e.g. : masquage des variables locales et globales par des variables locales.

Interpreter.java (suite)

```
@Override public Object visit(IASTInvocation iast, ILexicalEnvironment lexenv) throws EvaluationException
{
    Object function = iast.getFunction().accept(this, lexenv);
    if ( function instanceof Invocable ) {
        Invocable f = (Invocable)function;
        List<Object> args = new Vector<Object>();
        for ( IASTExpression arg : iast.getArguments() )
            args.add(arg.accept(this, lexenv));
        return f.apply(this, args.toArray());
    }
    else throw new EvaluationException("Cannot apply " + function);
}
```

Appel de primitive $e(e_1, e_2, \dots, e_n)$: (mais aussi, en ILP2, de fonctions utilisateur)

- évaluation de l'AST de e
f n'est pas nécessairement un nom de primitive, mais un nom de variable dans lequel on a stocké une primitive !
cf. : `let x = print in x(3)`
- vérification qu'il s'agit d'un objet callable **Invocable** (primitive, fonction, etc.)
- évaluation de chacune des expressions argument
- appel avec les valeurs d'arguments **args**, via la méthode **apply** d'**Invocable**
- le visiteur est également passé à **apply**, ce sera utile en ILP2 pour traiter les fonctions utilisateur

Interpreter.java (suite)

```
@Override public Object visit(IASTbinaryOperator iast, ILexicalEnvironment lexenv)
    throws EvaluationException
{
    Object leftOperand = iast.getLeftOperand().accept(this, lexenv);
    Object rightOperand = iast.getRightOperand().accept(this, lexenv);
    IASToperator operator = iast.getOperator();
    IOperator op = getOperatorEnvironment().getBinaryOperator(operator);
    return op.apply(leftOperand, rightOperand);
}
```

Comme les invocations, mais plus simple :

- le nombre d'arguments est fixé par le type de nœud AST
- recherche de l'opérateur dans l'environnement global
c'est garanti d'être un `IOperator` (test instanceof inutile)

Visite du programme complet (*top-level*)

Interpreter.java (suite)

```
public Object visit(IASTprogram iast, ILexicalEnvironment lexenv) throws EvaluationException {  
    try { return iast.getBody().accept(this, lexenv); }  
    catch (Exception exc) { return exc; }  
}
```

- évalue l'expression du programme
- retourne sa valeur, ou une exception est aussi transformée en objet retourné
- bien que s'appelant `visit`, ce n'est pas une méthode d'`IASTvisitor` !

InterpreterRunner.java (extrait)

```
public void interpretProgram(IASTprogram program) throws EvaluationException {  
    result = interpreter.visit(program, new EmptyLexicalEnvironment());  
    ...  
    System.out.println(" Value: " + result);  
    ...  
}
```

- point d'entrée (*top-level*) dans la **classe utilitaire** `InterpreterRunner` :
- interprète le programme dans un **environnement lexical vide**
- affiche la valeur de retour (entre autres)

Bibliothèque d'exécution

Invocable.java

```
public interface Invocable {  
    int getArity();  
    Object apply(Interpreter interpreter, Object[] argument) throws EvaluationException;  
}
```

Interface générale des valeurs ILP qui peuvent être appelés comme des fonctions :

- primitives, fonctions utilisateurs, méthodes, etc.
permet une bonne factorisation de nombreux cas!
- nombre arbitraire d'arguments
- **apply** prend les arguments **déjà évalués**
- si l'invocable est une fonction utilisateur, elle contient un AST qu'il faudra évaluer
⇒ l'interprète est également passé en argument pour faciliter l'évaluation

```
public interface IPrimitive extends Invocable {
    String getName();

    default Object erroneousApply() throws EvaluationException
    { throw new EvaluationException("Incorrect arity " + this.getName()); }

    default Object apply(Object arg1) throws EvaluationException { return erroneousApply(); }
    default Object apply(Object arg1, Object arg2) throws EvaluationException { return erroneousApply(); }
    ...

    @Override default Object apply(Interpreter interpreter, Object[] argument) throws EvaluationException {
        if ( argument.length == getArity() ) {
            switch (getArity()) {
                case 1: return apply(argument[0]);
                case 2: return apply(argument[0], argument[1]);
                ...
            }
        }
        else throw new EvaluationException("Wrong arity for operator " + this.getName());
    }
}
```

- classe utilitaire « squelette » pour différentes arités
- implante `apply` par dispatch à une invocation d'arité fixe
- il ne reste plus qu'à redéfinir dans l'interface la méthode `apply` d'arité souhaitée !

Primitives : exemple d'implantation

Print.java

```
package com.paracamplus.ilp1.interpreter.primitive;

public class Print extends UnaryPrimitive {

    private final Writer out;
    public Print(Writer out) { super("print"); this.out = out; }

    @Override public Object apply (Object value) throws EvaluationException {
        try { out.append(value.toString()); }
        catch (IOException e) { throw new EvaluationException(e); }
        return Boolean.FALSE;
    }
}
```

- affiche la valeur dans le canal `out`, spécifié à la création de la primitive
- hérite de `UnaryPrimitive`, qui définit `getArity` pour retourner 1
- qui hérite de `Primitive`, qui gère le nom de la primitive, fixé à la construction
- qui implante `IPrimitive`...
- `print` enrobe les exceptions Java en exceptions de l'interprète

IOperator.java

```
package com.paracamplus.ilp1.interpreter.interfaces;

public interface IOperator {
    String getName();
    int getArity();
    Object apply(Object ... argument) throws EvaluationException;
}
```

Très similaire à une primitive / fonction...

Operator est une implantation squelette, qui gère le nom de l'opérateur.

UnaryOperator et **BinaryOperator** spécialisent **Operator** en :

- fixant l'arité (1 ou 2)
- spécialisant la signature de **apply** (avec un ou deux arguments)

```
package com.paracamplus.ilp1.interpreter.operator;

public class Add extends BinaryOperator {
    public Add () { super("+"); }

    @Override public Object apply (Object arg1, Object arg2) throws EvaluationException {
        if ( arg1 instanceof String || arg2 instanceof String ) {
            return arg1.toString() + arg2.toString();
        } else if ( arg1 instanceof BigInteger ) {
            BigInteger bi1 = (BigInteger) arg1;
            if ( arg2 instanceof BigInteger ) {
                return bi1.add((BigInteger) arg2 );
            }
            else if ( arg2 instanceof BigDecimal ) {
                BigDecimal bd1 = new BigDecimal(bi1);
                return bd1.add( (BigDecimal) arg2 );
            }
            else throw new EvaluationException("Non numeric argument2");
        } else if ( arg1 instanceof BigDecimal ) {
            BigDecimal bd1 = (BigDecimal) arg1;
            if ( arg2 instanceof BigInteger ) {
                BigDecimal bd2 = new BigDecimal( (BigInteger) arg2 );
                return bd1.add(bd2);
            } else ... // Autres cas
        } else ... // Autres cas
    }
}
```

Initialisation des tables globales

GlobalVariableStuff.java

```
public class GlobalVariableStuff {  
    public static void fillGlobalVariables(IGlobalVariableEnvironment env, Writer out ) {  
        env.addGlobalVariableValue("pi", new BigDecimal("3.1415926535"));  
        env.addGlobalVariableValue(new ToString());  
        env.addGlobalVariableValue(new Print(out));  
        ...  
    }  
}
```

OperatorStuff.java

```
public class OperatorStuff {  
    public static void fillBinaryOperators (IOperatorEnvironment env) throws EvaluationException {  
        env.addOperator(new Add());  
        env.addOperator(new Multiply());  
        env.addOperator(new Subtract());  
        ...  
    }  
    public static void fillUnaryOperators (IOperatorEnvironment env) throws EvaluationException {  
        env.addOperator(new Minus());  
        ...  
    }  
}
```

- méthodes appelées par le *top-level* avant d'instancier [Interpreter](#)
- très facile d'ajouter des primitives et des opérateurs !

Tests

But du test :

- vérifier qu'un programme **vérifie sa spécification**
e.g. : pas d'exception au *top-level*, retourne bien le résultat attendu
- vérifier l'absence de régression (évolution du programme)

L'exécution d'un programme ILP (e.g., par l'interprète) a deux résultats :

- la valeur de retour
(un programme est une expression, et toute expression ILP a une valeur)
- le texte affiché sur la sortie standard par **print** et **newline**

⇒ nous allons vérifier les deux vis à vis des résultats attendus (spécification)

Format d'une banque de tests ILP : ensemble de triplets de fichiers

- **test.ilpml** : programme source ILP
- **test.result** : valeur de retour attendue
- **test.print** : sortie texte attendue

(cf. le répertoire **SamplesILP1**)

On lance le compilateur ou l'interprète sur chaque **test.ilpml**

pour vérifier que les sorties sont identiques à **test.result** et **test.print**.

JUnit 4 : *framework* de **test unitaire** utilisant les annotations Java.

Une classe de test doit :

- contenir un constructeur sans argument
- contenir une ou plusieurs **méthodes de test** :
 - annotées avec `@Test`
 - public et sans argument (donc bien différente d'une méthode `main...`)
- utiliser des assertions de la bibliothèque JUnit 4
`assertTrue`, `assertFalse`, `assertEquals`, etc.
- éventuellement contenir des méthode `@Before` et `@After`

Il est également possible de lancer un test sur une **famille de données** :

- une méthode `static @Parameters` fournit une liste de paramètres
- un constructeur avec argument est alors utilisé

(utile pour nos banques de tests...)

Test d'ILP : exemple de l'interprète

InterpreterTest.java (extrait)

```
@RunWith(Parameterized.class) public class InterpreterTest {
    @Parameters(name = "0") public static Collection<File[]> data() throws Exception
    { return InterpreterRunner.getFileList(samplesDirName, pattern); }

    public InterpreterTest(final File file) { this.file = file; }
    protected File file;

    @Test public void processFile() throws ... {
        InterpreterRunner run = new InterpreterRunner();
        configureRunner(run);
        run.testFile(file);
        run.checkPrintingAndResult(file);
    }

    public void configureRunner(InterpreterRunner run) throws EvaluationException {
        IASTfactory factory = new ASTfactory();
        run.setILPMLParser(new ILPMLParser(factory));
        ...
    }
}
```

La classe utilitaire `InterpreterRunner` automatise :

- l'extraction de la liste de fichiers sources (`getFileList`)
- l'appel à l'analyseur syntaxique et l'interprète (`testFile`)
- la comparaison entre le résultat attendu et obtenu (`checkPrintingAndResult`)

Le lancement du test peut se faire avec « *Run As* » puis « *JUnit Test* » sous Eclipse.

Pour chaque méthode `@Test`
et éventuellement chaque valeur de paramètre,
JUnit va :

- instancier la classe
- appeler les méthode `@Before`
- appeler la méthode de test
l'exécution du test s'arrête au premier échec d'assertion
- appeler les méthodes `@After`

Exécution en local : Eclipse affiche le nombre de tests qui n'ont aucune erreur.

Intégration continue des TME :

à chaque *push* d'un TME, lors d'un rendu, les tests sont lancés automatiquement sur un serveur et les résultats visibles sur l'interface GitLab !

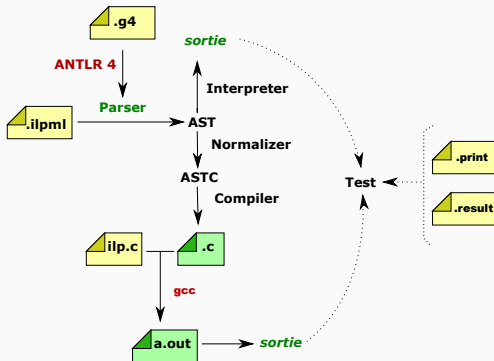
```
image: dlp

stages:
- cours
- tme

ILP1:
  stage: cours
  script: >
    /home/dlp/run.sh -id ILP1 -name ILP1
    com.paracampus.ilp1.interpreter.test.InterpreterTest
    com.paracampus.ilp1.compiler.test.CompilerTest
  artifacts:
    reports:
      junit: report.xml
```

Fichier de configuration `.gitlab-ci.yml` à la racine du répertoire

- format **YAML** ; attention aux espaces !
- exécuté à chaque *git push* par le serveur, dans un conteneur **docker**
image système virtuelle, bien isolée du serveur, contenant juste Linux, ANTLR 4 et Java
- le fichier `report.xml` fourni est utilisé par GitLab pour l'affichage des résultats
- à enrichir avec des nouvelles entrées sur le modèle de celle pour **ILP1**
à chaque TME, en spécifiant les **nouvelles classes de test**



Nous avons vu :

- les visiteurs
- l'interprète (un visiteur)
- la représentation des valeurs (Java)
- l'environnement (local, global)
- la bibliothèque d'exécution (opérateurs, primitives)
- le test automatisé avec **JUnit 4**

Prochain cours : compilateur ILP1 vers C en Java, bibliothèque d'exécution en C.